

```
In [1]: import nltk, random
nltk.download('twitter_samples')
from nltk.corpus import twitter_samples
```

```
[nltk_data] Downloading package twitter_samples to C:\Users\Ujjwal
[nltk_data] Karkk\AppData\Roaming\nltk_data...
[nltk_data] Package twitter_samples is already up-to-date!
```

```
In [2]: pos = twitter_samples.strings('positive_tweets.json')
neg = twitter_samples.strings('negative_tweets.json')
```

```
In [3]: data = [(t, 1) for t in pos] + [(t, 0) for t in neg]
random.shuffle(data)
```

```
In [4]: train, test = data[:8000], data[8000:]
```

```
In [5]: import re
def clean(t):
    # replace https with " "
    t = re.sub(r"http\S+@\w+", " ", t)
    t = re.sub(r"^[a-zA-Z\s]", " ", t)
    return t.lower().split()
```

```
In [6]: from collections import Counter
counts = Counter(w for t, _ in train for w in clean(t))
```

```
In [7]: vocab = {"<pad>": 0, "unk": 1}
for w, _ in counts.most_common(8000):
    vocab[w] = len(vocab)
```

```
In [8]: import torch
from torch.utils.data import DataLoader, TensorDataset
```

```
In [9]: def encode(t, maxlen=30):
    ids = [vocab.get(w, 1) for w in clean(t)][:maxlen]
    return ids + [0]*(maxlen - len(ids))
```

```
In [10]: Xtr = torch.tensor([encode(t) for t, _ in train])
ytr = torch.tensor([1 for _, 1 in train])
```

```
In [11]: train_loader = DataLoader(TensorDataset(Xtr, ytr), batch_size = 32, shuffle=True)
```

```
In [12]: import torch.nn as nn
```

```
In [13]: class SentimentLSTM(nn.Module):
    def __init__(self, vocab_size, emb=64, hidden=128):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, emb)
        self.lstm = nn.LSTM(emb, hidden, batch_first=True)
        self.fc = nn.Linear(hidden, 2)

    def forward(self, x):
        x = self.embedding(x)
        x, _ = self.lstm(x)
        return self.fc(x[:, -1]) # Last hidden memory
```

```
In [14]: device = "cuda" if torch.cuda.is_available() else "cpu"
         print(f"Using {device} device")
```

Using cuda device

```
In [15]: lstm = SentimentLSTM(len(vocab)).to(device)
         optimizer = torch.optim.Adam(lstm.parameters(), weight_decay = 0.0001)
         loss_fn = nn.CrossEntropyLoss()
```

```
In [18]: for epoch in range(1500):
         lstm.train()
         for xb, yb in train_loader:
             xb, yb = xb.to(device), yb.to(device)
             pred = lstm(xb)
             loss = loss_fn(pred, yb)
             optimizer.zero_grad() # torch uses grad from previous steps, this clear those
             loss.backward()
             optimizer.step()

         if epoch % 100 == 0:
             print(f"Epoch {epoch} Loss {loss.item()}")
```

```
Epoch 0 Loss 0.6933854222297668
Epoch 100 Loss 0.6941050291061401
Epoch 200 Loss 0.6934892535209656
Epoch 300 Loss 0.6933952569961548
Epoch 400 Loss 0.6931530833244324
Epoch 500 Loss 0.6944977641105652
Epoch 600 Loss 0.6925996541976929
Epoch 700 Loss 0.6938742399215698
Epoch 800 Loss 0.6929667592048645
Epoch 900 Loss 0.6931494474411011
Epoch 1000 Loss 0.6933999061584473
Epoch 1100 Loss 0.6928544044494629
Epoch 1200 Loss 0.6922163367271423
Epoch 1300 Loss 0.6934161186218262
Epoch 1400 Loss 0.6930317878723145
```

```
In [19]: Xte = torch.tensor([encode(t) for t, _ in test])
         yte = torch.tensor([l for _, l in test])

         lstm.eval()
         with torch.no_grad():
             acc = lstm(Xte.to(device)).argmax(dim=1).eq(yte.to(device)).sum().item() / len
         print(acc)
```

0.5015